

Integrating AI and Structural Biology: LSTM Model for Protein Secondary Structure Prediction Based on Amino Acid Sequences

Guilherme Henriques, Iago Vale, Rafael Gaspar

Faculty of Science and Technology of the University of Coimbra

Abstract. The abstract examines the utilization of a Long Short-Term Memory (LSTM) model, which is a type of Recurrent Neural Network (RNN), for the prediction of the Secondary Structure of proteins based only on their amino acid sequences. This study explores the efficacy of LSTM architectures in the extraction of meaningful patterns and features from the information present within the amino acid sequences of proteins to accurately predict the corresponding Secondary Structures. Throughout this project various LSTM configurations and parameters are tested and investigated, including data preprocessing techniques, layer structures, training methodologies and result observations. This project aims to assess the predictive capabilities of LSTM models and their potential in Protein Secondary Structure prediction tasks, contributing to our knowledge of Structural Biology precision and Structural assessment.

Keywords: Machine Learning · Artificial Intelligence · Computational models · Protein · Secondary Structure · Long Short-Term Memory

1 Introduction

In this work, the focus is on machine learning, a subfield of artificial intelligence (AI) that involves developing algorithms and statistical models to enable computers to learn from data and make predictions. Machine learning models are trained on a set of data and work by finding patterns, relationships, and trends within that data (Matthew et al., 2020). Machine learning can be a powerful tool for analysing and interpreting complex biological data, especially in the field of genomic and proteomic data analysis (David et al., 2022). By studying large sets of proteomics data, such as protein sequences, machine learning can identify patterns associated with specific structures or traits that would otherwise be difficult to detect using traditional methods.

Proteins are a diverse class of biological molecules that are categorized as structural and functional units of the cell. They are formed by polymers made of amino acids connected by peptide bonds. Each protein is unique, depending on their amino acid's chains, with that being related, more specifically, to the position that each amino acid occupies within its sequence, number of amino acids, physical and chemical interactions, among other factors (Jain et al., 2022).

When we talk proteins and their structure, most of the times, we categorize them as complex, having different levels of organization. The primary structure refers to the number and sequence of the amino acids within the polypeptide chain.

The secondary structure refers to the general 3D shape of the local protein segments, being related to the spatial arrangement of amino acid residues that are neighbours in their primary structure (Guzzo, A., 2019). While it does not specify the atomic level coordinates and positions in 3D space, it is defined by patterns of bonds between the Amide and Carboxylic groups. This type of structure also contributes significantly to general protein structure and functions (Gu et al., 2023), being commonly characterized by helix (H), sheet (E) and coil (C), but DSSP program (Kabsch & Sander, 1983) proposed a finer characterization extending the states who are 3_{10} helix (G), α -helix (H), π -helix (I), β -strand (E), bridge (B), turn (T), bend (S) and others (C) (Zhang et al., 2018).

The tertiary structure describes the complete 3D architecture of the protein, and the quaternary structure applies only to proteins formed by two or more polypeptide chains, referring to the spatial arrangement of the chains and the links between them (Blanco & Blanco, 2017).

In this project, we used a Long Short-Term Memory (LSTM) model, which is a type of Recurrent Neural Network (RNN). This type of model was designed to overcome limitations of traditional RNNs, being first introduced during the end of the 90s, presenting a unique architecture containing the Forget, Input and Outputs gates, which can regulate the flow of information in the model. These gates, along with the cell state, enable the LSTM to retain the more important/relevant information over long sequences), while discarding the irrelevant data (Houdt et al., 2020). These networks tackle the vanishing gradient issue of conventional RNNs by incorporating a memory cell and gating mechanisms to control the flow of information (Hochreiter & Schmidhuber, 1997). The forget gate determines which information to remove from the cell state, whereas the input gate specifies what new information to save (Gers et al., 2000). The cell state functions like a highway for maintaining long-term dependencies, facilitating effective information transmission. The output gate regulates the information transmitted to the subsequent layer or time step (Graves, 2013).

LSTMs are widely used in tasks involving sequential data, examples of such are Natural Language Processing (NLP), Time-Series Processing (TSP), and Automatic Speech Recognition (ASR), these are good examples as they work by effectively capturing their temporal patterns. However, they can be very computationally intensive/taxing and struggle when it comes to very long sequences, in these cases, transformer-bases models often outperform them. Despite these challenges, LSTM models remain a cornerstone of Deep Learning for sequence modelling due to their ability to handle complex temporal relationships (Lindemann et al., 2021). This type of model has shown promising results when it comes to predicting the secondary structure of proteins from sequences and can be used in conjunction with other models to obtain the best possible accuracy for the intended purpose (Cheng et al., 2020; Enireddy et al., 2022; Zhao et al., 2024).

This work aims to create a computational model using Machine Learning algorithms to learn patterns and features associated with the Secondary Structure of proteins through mathematical and algorithmic approaches. By utilizing a mixed LSTM model and its predictions, statistical information, and the simulation of different scenarios, insights into the underlying mechanisms of protein structures can be gained.

2 Methods

2.1 Model Architecture

This study adopted a Long Short-Term Memory (LSTM) model, a class of Recurrent Neural Networks (RNN), which is specifically useful when dealing with sequential data, designed to effectively capture and handle complex temporal patterns and relationships. The architecture of our model is depicted in the Figure 1, it is composed of an input layer, which consists in the amino acid sequences of our chosen proteins, followed by three LSTM layers, which are crucial for protein secondary structure prediction, not only because they excel at capturing long-range dependencies and sequential patterns in amino acid sequences, but also because proteins often rely on distant residue interactions to form secondary structures like alpha-helices and beta-sheets, and LSTM layers are able to portray these relationships effectively, improving its predictive accuracy. At the end we have an output dense layer with three exits, one for each of the three main secondary structures.

The architecture changed a lot throughout the project, originally in hopes of preventing the overfitting, functions and layers were added. While towards the end, to address the underfitting other measures were required, leading to the results shown in the following sections of the article. After implementing the model, it will need to be trained by feeding it data, which will have to be pre-processed first, and then tested to evaluate its performance. There are many factors and methods for evaluating a model's performance, but the chosen metrics depend greatly on the specific problem and the model's target. In this case, the accuracy and loss of the model will be checked to determine how accurate and efficient it is at predicting the secondary structure of our proteins, and we will also be evaluating metrics like the F1 score (harmonic mean of precision and recall), and the AUC (Area Under the Curve) more by the end of the project.

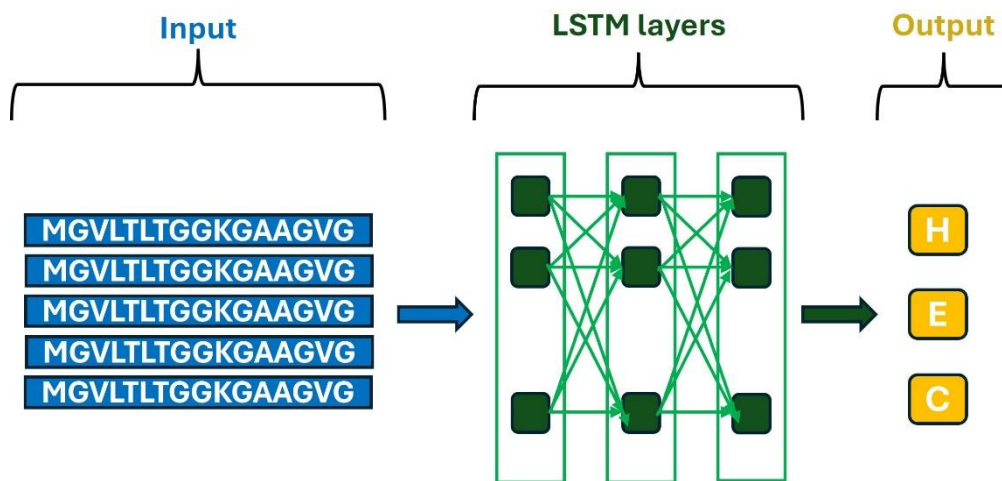


Fig. 1. Long Short-Term Memory structure: the designed architecture efficiently extracts patterns from protein's amino acid sequences to predict their secondary structures.

2.2 Dataset

The dataset that was used in this project was the latest CASP12 (Critical Assessment of Techniques for Protein Structure Prediction) text file, which is a widely used benchmark for evaluating methods in protein structure prediction. It contains a variety of different information, but the essential for us was the target information, including protein ID, sequence length, primary amino acid sequences, predicted 3D coordinates, confidence scores, and secondary structure annotations (meaning the predicted secondary structures: alpha-helices, beta-sheets, coils and other sub-structures). The dataset was derived from the CASP competition, organized by the Protein Structure Prediction Centre. It provides high-quality target datasets by collaborating with experimental labs to make newly discovered protein structures available for blind prediction. This organization aims to advance the field of computational biology and bioinformatics by rigorously and constantly testing and comparing the accuracy of different protein prediction algorithms and setting new benchmarks for the field.

2.3 Pre-Processing

After gathering our dataset, we had to start working on it to adapt it better to our needs. First, we started by removing the columns that weren't exactly required for our model, ending up with the protein ID, Amino Acid Sequence and the Secondary Structures, which had a prediction for the main secondary structures as well as their sub-structururs for each amino acid.

Then we selected only a certain number of proteins, as the model had hundreds of thousands, ending up training our model with usually 10 thousand proteins. Out of those 10 thousand we created a function that kept only the ones that had at least 90 characters (amino acids). For that decision we first created a Histogram, that showed the distribution of the different sequences and the number of characters each one had. This step allowed us to see how some proteins had a substantially smaller sequence length than others, and that the median was around 300 characters, which was very helpful later on, when it came to padding the sequences, something that will be mentioned in more detail later in the paper.

After this first initial stage, we started applying the pre-processing steps, first we had our amino acid sequences, which were used to predict their secondary structures. The prediction of the secondary structures works by replacing each individual amino acid with a letter, where each letter is a representation of one of the seven types of main/sub structures, this is a crucial step because it will help not only us but the model in understanding the folding patterns and functions of our proteins, which will further improve the confidence of the decisions that the model will be making.

Then we Mapped the secondary structures, this consisted in numerically encoding the secondary structure sequences by assigning each individual letter a number (0 = 'H' | 1 = 'E' | 2 = 'C'), this is also an essential part of the pre-processing, as our model will be intaking numerical data and not categorical data. The last step was the One Hot Encoding, which consists in converting the numerical sequences obtained in the previous process

into one hot encoding vectors, this is done by replacing each number by a representative cell ($[1,0,0] = 0$ | $[0,1,0] = 1$, $[0,0,1] = 2$). This is the most important step as this type of encoding will help the model with its output probabilities. With this type of representation, each amino acid will be represented more individually, allowing the model to better distinguish each amino acid, making it more thorough and precise, making the model overall more robust.

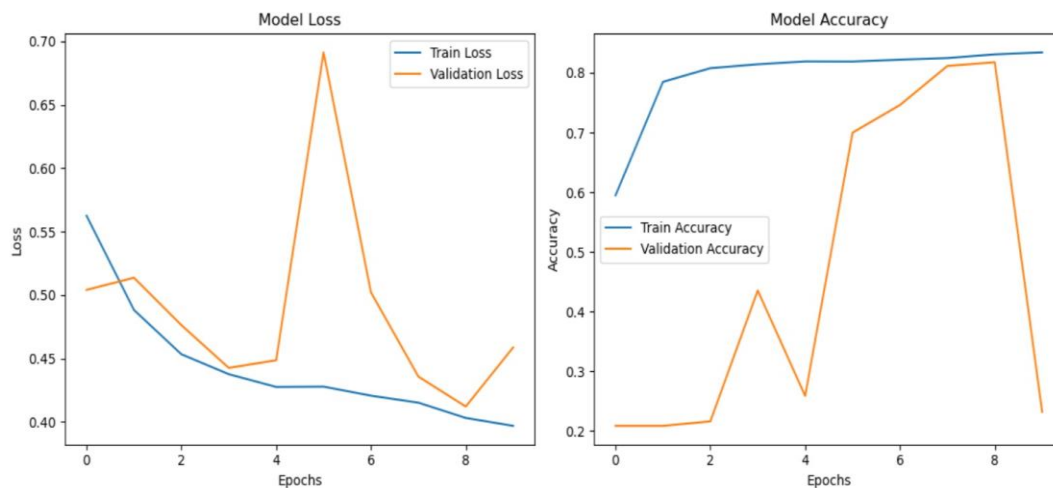
Before running our code, now that the output is already prepared, we still had to prepare the input for the LSTM model. For that, we had to convert each amino acid of each sequence from a letter to a suitable format for the model, and for that we asked ChatGPT to create a dictionary of all the amino acids and associate them to a vector (like the one hot encoding ones). Then we just replaced each letter with its corresponding representation.

3 Experimental Analysis and Results

The model development involved multiple stages of trial and error, from tests lasting a couple of hours to tests lasting 15 hours and up. After each test, the results were analysed so we could identify potential issues or factors affecting the model's performance with the intent of finding a countermeasure for those same issues, therefore being able to continuously improve our model's efficacy. The initial code was simpler and less structured, with subsequent parameter tuning and the addition of layers and techniques, the model's precision improved significantly. Visual representations, such as graphs, were used to illustrate the model's outputs and performance more effectively, allowing for a better visualization of the model evolution. Below, the results are explained based on different stages of model refinement.

3.1 Initial Model Testing

In the first model, we started by experimenting with our code to guarantee that all the pre-processing steps were working accordingly in a chain, and then feeding that refined data into our model architecture. Initially we started off simple, we used padding to ensure all sequences had the same length, this function works by limiting the length of all sequences and adding zeros to the end of the shorter ones until they reach the defined capped length. After doing so, we ran our code for 10 epochs only to, once again, ensure everything was working accordingly. As we can see on the results shown in the Figure 2, the model seems to be working, however the accuracy seen in the test (81%) doesn't seem too fitting in comparison to the learning curves. Although the model only trained for 10 epochs, it showed some problems when it came to data conversion, as expected, as we had basically only that one LSTM layer. Based on these results, next steps consisted in exploring solutions, such as adjusting parameters and modifying the model's architecture to avoid both overfitting and underfitting.



Test Loss: 0.4122, Test Accuracy: 0.8176

Fig. 2. Testing the accuracy and loss values of the proposed model over a total of 10 epochs, with a graphical representation for the results showing a curve for the loss and accuracy of both the training and testing set, also represented beneath the graphics the test values for both Loss and Accuracy.

3.2 Improved Model with Early Stopping

Besides training our model with more epochs now, we also did a major step forward by adding two more LSTM layers to help address overfitting, which we were seeing in previous tests, and an Early Stopping function. Having more LSTM layers allows the model to learn increasingly complex patterns, capturing both short and long-term dependencies in the data, and by stacking these, we can generalize the data more, and reduce the reliance of the model on one single layer. The Early Stopping was also added to prevent overfitting, as it is responsible for stopping the training process once the model's performance stops improving, helping avoid unnecessary training epochs and help save time. By looking at Figure 3, we can see the results obtained in this stage of the project. By having more epochs during training, we can now have a better grasp on what is happening with our model's performance. We see that the accuracy is pretty good at around 83%, showing better learning curves for both loss and accuracy. However, overfitting was still present, which meant that further refinements were needed.

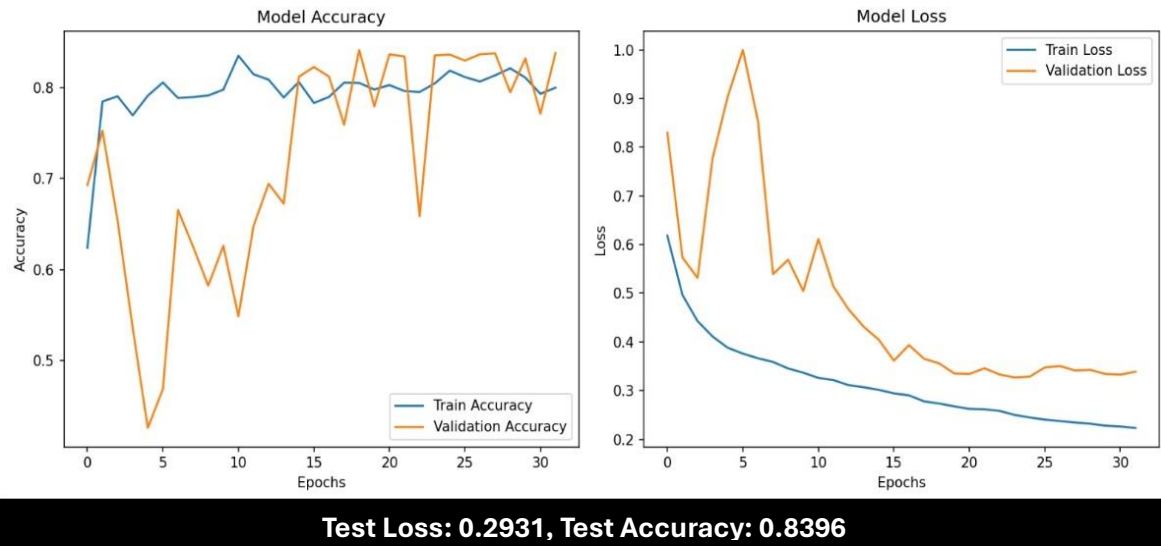


Fig. 3. Testing the accuracy and loss values of the proposed model over a total of 30 epochs out of 50 due to the early stopping, with a graphical representation for the results showing a curve for the loss and accuracy of both the training and testing set, also represented beneath the graphics the test values for both Loss and Accuracy.

3.3 Further Refinements and Introduction of New Functions

By looking at past tests and the latest results we had achieved, we thought keeping the same three LSTM architecture was a good foundation, however, more regularization and optimization was needed. For that, we added Dropouts and normalization to every layer, an optimizer “Adam”, as well as a learning rate scheduler. The Dropouts are a type of regularization technique that prevents overfitting by “dropping” a fraction of the neurons during each training step, which results in the model becoming less reliant on specific neurons and avoiding the memorization of the training data. The Batch Normalization stabilizes training and helps with data conversion allowing the model to generalize better. The optimizer works hand to hand with the learning rate, by efficiently adjusting it for each parameter, leading to faster convergence, which prevents the model from getting “stuck”.

The learning rate scheduler was added because we saw the model was reaching stagnation points, and how this technique works is, it decreases the learning rate during training over time, allowing for larger updates initially and smaller, more refined adjustments later, once again improving data conversion. Looking at Figure 4, we see a decrease of around 3% in accuracy, a near “perfect” loss learning curve, but a not so good-looking accuracy growth. Now, the graphical representation of the testing set indicated that overfitting, while reduced, was still present.

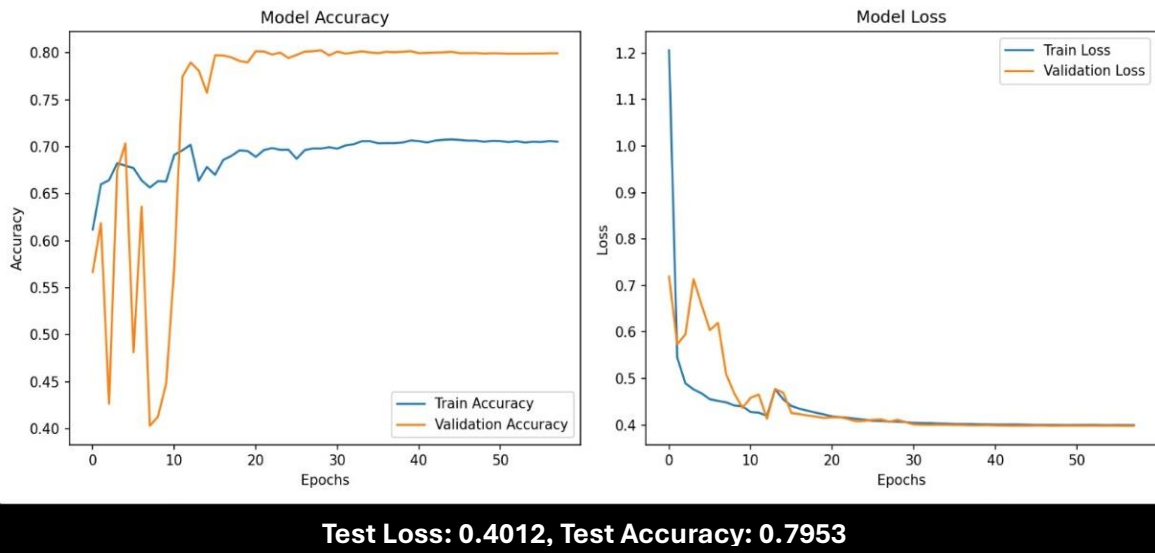
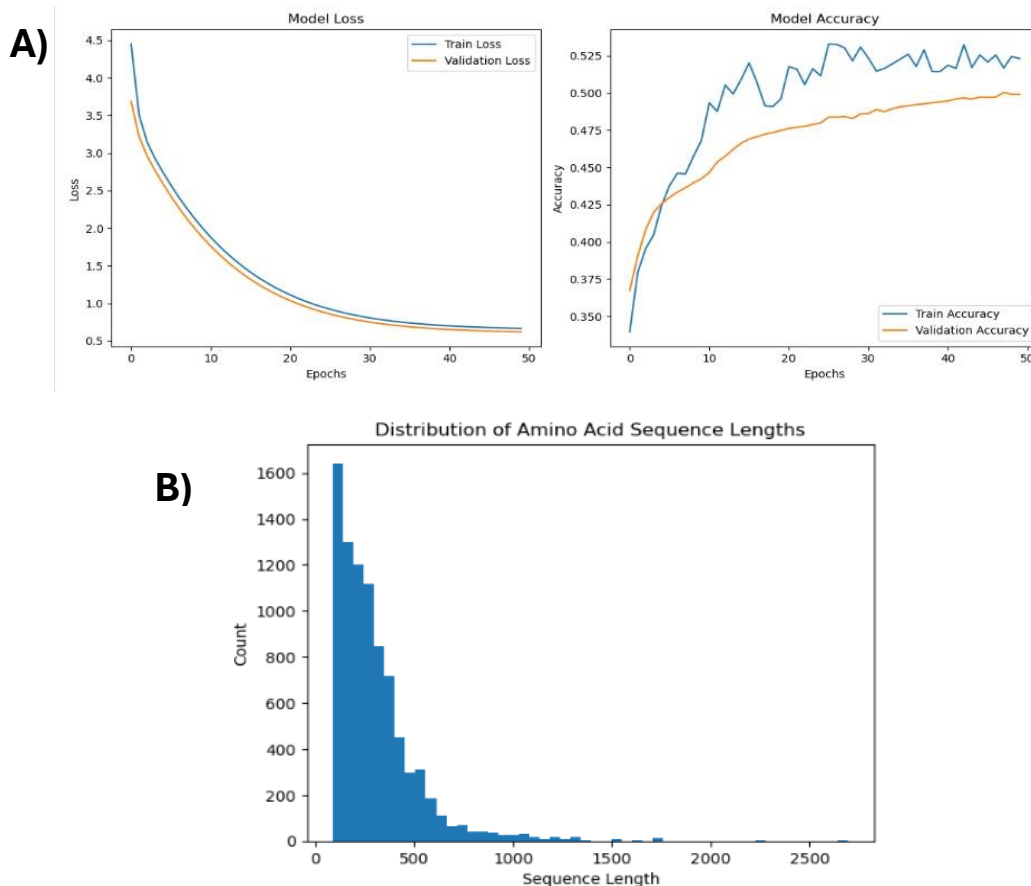


Fig. 4. Testing the accuracy and loss values of the proposed model over a total of 50 epochs, with a graphical representation for the results showing a curve for the loss and accuracy of both the training and testing set, also represented beneath the graphics the test values for both Loss and Accuracy.

3.4 Overfitting Reduction and Data Regularization

At this point, the overfitting was our main focus, and we had to take some steps backwards in order to be able to take some forward. We decided to test our architecture by adding Convolutional layers, as they are well known for having great feature extraction, which was something we were struggling with, especially in early training epochs. Some changes were also made to our Learning Rate Scheduler, creating a “warm up”, where we would, as the name suggests, gradually increase the learning rate during the initial epochs, and then keeping the same format as usual. We also kept playing around with the layer dimensions, increasing them in hopes of making them capture more complex patterns and avoid overfitting even more. However, what we thought was a real game changer, was the fact that, we noticed through the creation of a Histogram plot, how our dataset had a majority of sequences with a median of 300 characters (amino acids) and a pretty significant of outliers that either had way more characters or way less in comparison to the median. For that we created a function where we would limit the minimum length allowed for each sequence and, therefore, also fix the padding to prevent unnecessary zeros within each sequence, which were sought to be impacting our model’s performance negatively. Figure 5 shows the big jump we had forward when it came to the actual performance of the model, with the growth of the learning curves, being way less “spikey” and showing a significant reduction in overfitting but can’t forget the step back in terms of the accuracy, which decreased to near 50%. Although the model showed a reduction in the accuracy, at least, these changes led to a more consistent performance.

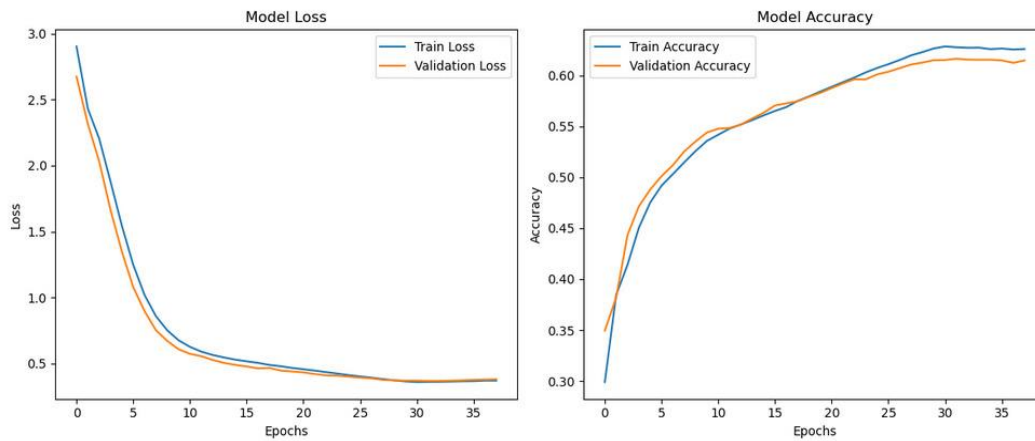


Test Loss: 0.6200, Test Accuracy: 0.4990

Fig. 5. Testing the accuracy and loss values of the proposed model over a total of 50 epochs. **A)** graphical representation for the results showing a curve for the loss and accuracy of both the training and testing set; **B)** graphical representation of the correlation between the length of each protein's sequence and the number of proteins with those same lengths, through a Histogram.

3.5 Model Modifications and Attention Mechanism

A model checkpoint function was added so only the best performing epochs get selected throughout the training process. Our learning rate was changed into a Cyclic Learning rate, which alternates between a lower and upper defined learning rate value in hopes of helping the model further reduce overfitting by escaping stagnation, which ended up helping overcome the early plateau observed with early stopping. Another major step was the Multi Headed Attention Mechanism that was added, that works by allowing the model to focus on different parts of the input sequences simultaneously, capturing various patterns within the data, improving the model's ability to learn more complex dependencies. Due to these changes, we were able to achieve a 10% increase in accuracy getting up to around 62%, with an even better model performance all around, as you can see reflected in the learning curves from Figure 6.



Test Loss: 0.3666, Test Accuracy: 0.6161

Fig. 6. Testing the accuracy and loss values of the proposed model over a total of 35 epochs out of 50 due to the early stopping, with a graphical representation for the results showing a curve for the loss and accuracy of both the training and testing set, also represented beneath the graphics the test values for both Loss and Accuracy.

3.6 Additional Techniques and Final Model Adjustments

The last adjustments consisted solely in trying to improve the accuracy values, as the results were already showing a good overall performance. After numerous tests and experiments, we came to the conclusion that the hybridization that we were trying with the Convolutional layers wasn't really having a huge impact, so our last and best model ended up having only three LSTM layers with a gradual decrease in layer size, the learning rate was changed from cyclic to a scheduler that would reduce the learning rate once it hit a plateau for a selected number of epochs, coming to the conclusion that a simpler model was in fact achieving better results overall, as we can see by looking at Figure 7, where we see that there was an exponential jump in accuracy from the previous results shown in the article, increasing to a value of around 80%. Although we do see a resemblance of overfitting, it isn't a huge concern, having in mind that the overall performance is stable, with the learning curves being represented as expected. We also decided to measure the F1 score, which indicates that the model is correctly identifying relevant results 76% of the times, while minimizing false positives and false negatives. In terms of the AUC, the score obtain means that the model has an 89% chance of distinguishing between different classes indicating an excellent performance, while the graph also validates this value, showing that overall, the model generalizes well without underfitting or overfitting.



Fig. 7. Testing the accuracy and loss values of the proposed model over a total of 50 epochs, with a graphical representation for the results showing a curve for the loss, AUC and accuracy of both the training and testing set, also represented beneath the graphics the test values for Loss, AUC, F1 score and Accuracy.

3.7 Final Model Adaptation and Comparison

Having in mind our dataset has prediction sequences for not only the three main secondary structures but also their four sub-structures, our last step consisted in adapting our best performing model by executing a slightly different data pre-processing, where we now represent every single structure type, without grouping the sub-structures to their “main” structure, increasing the number of outputs from three to seven. Looking at Figure 8, we can see that graphically it seems like the model is being able to converge the data correctly, with good learning curves for Loss, Accuracy and AUC, however, when it comes to the F1 score it was low (34%) in comparison to the results obtained in the previous test. The accuracy was also low with only 48%, which was somehow expected as we didn’t exactly create an architecture fully dedicated to these sub-structures in question, rather using the best previously obtained model due to time restrictions. The 90% AUC score obtained means that the model has an excellent performance in distinguishing between different classes, while the graph also validates this value, showing that overall, the model generalizes well without overfitting, however in terms of underfitting we could presume that, due to the low accuracy value we could be experience some degree of it, which is something we will have to address in further experiments. But in general, having all these factors mentioned before in mind, the results weren’t bad.

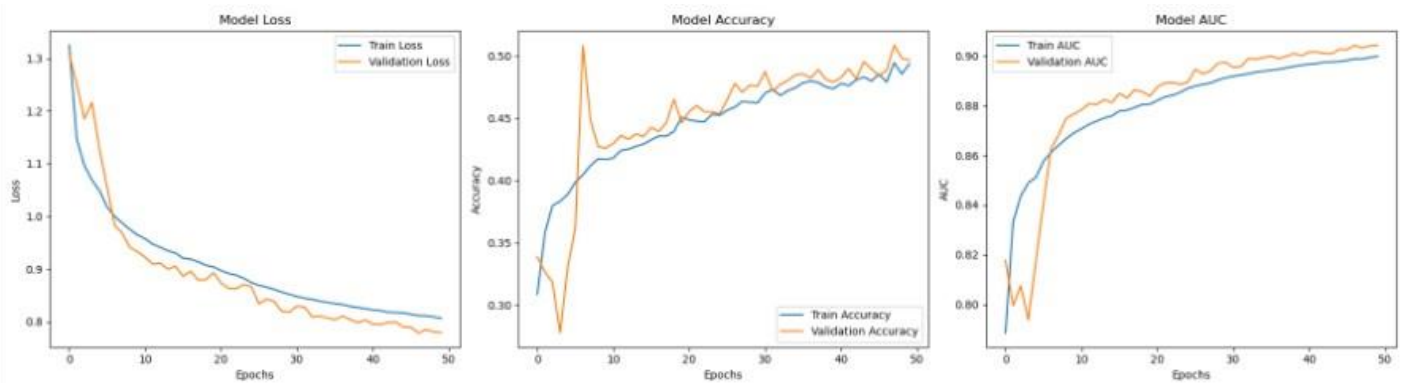


Fig. 8. Testing the accuracy and loss values of the proposed model over a total of 50 epochs, with a graphical representation for the results showing a curve for the loss, AUC and accuracy of both the training and testing set, also represented beneath the graphics the test values for Loss, AUC, F1 score and Accuracy.

4 Conclusion

In this project, we successfully developed and reined a Long Short-Term Memory (LSTM) model for protein secondary structure prediction based only on their amino acid sequences. Through a series of iterative adjustments, including changes in data portioning, architecture, and regularization techniques, our model achieved a significant improvement in accuracy, reaching near 80%. The main factor/aspect of this project was applying a deep learning, specifically LSTMs, to the field of Structural Biology for the prediction of the three main secondary structures, but also ending up also doing it for the remaining four sub-structures. By leveraging its efficiency in capturing and handling complex temporal patterns and relationships, we were able to identify patterns within the protein's amino acid sequences that tradition methods might overlook. The integration of Dropout layers, learning rate schedules, early stopping and attention mechanisms further optimized our model's performance, reducing overfitting and enhancing generalization to unseen data.

Despite the achievements showcased throughout this article, this project also shows some limitations that we had throughout this semester, more specifically related to our computational power which is directly proportional to the time we had available. Due to long testing processes the number of precise and impactful improvements was also affected, resulting in a lesser amount of them as we would've liked.

In terms of future improvements, there is a whole world of possibilities with numerous functions and techniques that can be tested and implemented in hopes of improving accuracy and overall performance of the model. Some things that could be promising are, per example, diving a little bit more into the hybrid architectures, where we

could mix our present LSTM architecture with more convolutional layers and explore this hybridization more, or with Transformers, to help with capturing both sequential and spatial features. Other improvements could be to work on testing more and different hyper-parameter tunings and combinations, improving other techniques such as the types of attention mechanisms and learning schedules. The last main thing we could really improve on is put some more thought and time into the LSTM architecture for the seven secondary structure types, which we weren't able to specifically design a model for, ending up, due to time constraints, using our best performing model for the three main secondary structures for this reasonably different case, which could have direct consequences in the models performance and accuracy.

References

- 1 Blanco, A., & Blanco, G. (2017). Proteins. Em *Medical Biochemistry* (pp. 21–71). Elsevier. <https://doi.org/10.1016/B978-0-12-803550-4.00003-3>
- 2 Cheng, J., Liu, Y., & Ma, Y. (2020). Protein secondary structure prediction based on integration of CNN and LSTM model. *Journal of Visual Communication and Image Representation*, 71, 102844. <https://doi.org/10.1016/j.jvcir.2020.102844>
- 3 David Sidak, Jana Schwarzerová, Wolfram Weckwerth, and Steffen Waldherr. Interpretable machine learning methods for predictions in systems biology from omics data. *Frontiers in Molecular Biosciences*, 9:926623, 2022. <https://doi.org/10.3389/fmolb.2022.926623>
- 4 Enireddy, V., Karthikeyan, C., & Babu, D. V. (2022). OneHotEncoding and LSTM-based deep learning models for protein secondary structure prediction. *Soft Computing*, 26(8), 3825–3836. <https://doi.org/10.1007/s00500-022-06783-9>
- 5 Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10), 2451–2471. <https://doi.org/10.1162/089976600300015015>
- 6 Graves, A. (2014). *Generating sequences with recurrent neural networks* (No. arXiv:1308.0850). arXiv. <https://doi.org/10.48550/arXiv.1308.0850>
- 7 Gu, J., Xu, Y., & Nie, Y. (2023). Role of distal sites in enzyme engineering. *Biotechnology Advances*, 63, 108094. <https://doi.org/10.1016/j.biotechadv.2023.108094>
- 8 Guzzo, A. (2019). Data storage and representation. Em *Encyclopedia of Bioinformatics and Computational Biology* (pp. 117–125). Elsevier. <https://doi.org/10.1016/B978-0-12-809633-8.20411-1>
- 9 Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- 10 Houdt, G., Mosquera, C., & Nápoles, G. (2020). A review on the long short-term memory model. *Artificial Intelligence Review*, 53(8), 5929–5955. <https://doi.org/10.1007/s10462-020-09838-1>
- 11 Jain, M., Amera, G. M., Muthukumaran, J., & Singh, A. K. (2022). Insights into biological role of plant defense proteins: A review. *Biocatalysis and Agricultural Biotechnology*, 40, 102293. <https://doi.org/10.1016/j.bcab.2022.102293>

- 12 Kabsch, W., & Sander, C. (1983). Dictionary of protein secondary structure: Pattern recognition of hydrogen-bonded and geometrical features. *Biopolymers*, 22(12), 2577–2637. <https://doi.org/10.1002/bip.360221211>
- 13 Lindemann, B., Müller, T., Vietz, H., Jazdi, N., & Weyrich, M. (2021). A survey on long short-term memory networks for time series prediction. *Procedia CIRP*, 99, 650–655. <https://doi.org/10.1016/j.procir.2021.03.088>
- 14 Matthew Nagy, Nathan Radakovich, and Aziz Nazha. Machine learning in oncology: what should clinicians know? *JCO Clinical Cancer Informatics*, 4:799–810, 2020. <https://doi.org/10.1200/CCI.20.00049>
- 15 Zhang, B., Li, J., & Lü, Q. (2018). Prediction of 8-state protein secondary structures by a novel deep learning architecture. *BMC Bioinformatics*, 19(1), 293. <https://doi.org/10.1186/s12859-018-2280-5>
- 16 Zhao, L., Li, J., Zhan, W., Jiang, X., & Zhang, B. (2024). Prediction of protein secondary structure by the improved TCN-BiLSTM-MHA model with knowledge distillation. *Scientific Reports*, 14(1), 16488. <https://doi.org/10.1038/s41598-024-67403-0>